atelier-stagiaires

Atelier 6 : prérequis du Sprint 3 (Quick Annonces) · PHP, MySQL Aiven, Cloudinary et Vercel Ali EL Habib · OFPPT Tanger · Développement Digital

Application de gestion des stagiaires :

- **Page 1 · Inscription** (`index.html`) : nom, prénom, ville et photo. Le serveur envoie la photo à Cloudinary, puis enregistre le stagiaire dans MySQL.
- **Page 2 · Liste** (`liste.html`) : lit les stagiaires depuis Aiven et affiche leurs photos hébergées sur Cloudinary.

Architecture

```
Navigateur (index.html / liste.html)
        │  fetch()
        ▼
Vercel : fonctions PHP (api/inscrire.php, api/liste.php)
        │                          │
        ▼                          ▼
Aiven MySQL                 Cloudinary
(nom, prénom, ville,        (fichiers image,
 URL de la photo)            URL sécurisée https)
```

- **Aiven** stocke les données relationnelles.
- **Cloudinary** stocke les fichiers image.
- **PHP** relie le formulaire, la base de données et le service d’images.
- **Vercel** sert les pages HTML statiques et exécute les fonctions PHP (`vercel-php@0.9.0`).
- **GitHub** conserve le code et déclenche le déploiement automatique sur Vercel.

Structure du projet

```
atelier-stagiaires/
├── index.html          Formulaire d'inscription
├── liste.html          Affichage des stagiaires
├── style.css           Style commun aux deux pages
├── composer.json       Dépendance Cloudinary (cloudinary/cloudinary_php ^3.0)
├── vercel.json         Runtime PHP pour api/*.php
├── .gitignore          vendor/ et fichiers .env
├── api/
│   ├── inscrire.php    Écriture : photo → Cloudinary, stagiaire → MySQL
│   └── liste.php       Lecture : SELECT des stagiaires, réponse JSON
├── config/
│   ├── db.php          Connexion PDO à Aiven (TLS avec ca.pem)
│   └── http.php        Réponses JSON et messages d'erreur
├── certs/
│   └── ca.pem          Certificat CA d'Aiven (à ajouter)
└── sql/
    └── schema.sql      CREATE TABLE stagiaires
```

Mise en place

1. Aiven : base MySQL

1. Créer un service MySQL dans la console Aiven, puis attendre qu'il soit *Running*.
2. Relever : hostname, port, base (`defaultdb`), utilisateur (`avnadmin`), mot de passe.
3. Télécharger le **CA certificate** et l'enregistrer dans `certs/ca.pem`.
4. Créer la table (depuis `C:\wamp64\bin\mysql\mysql8.2.0\bin`) :

```
mysql -h <hostname> -P <port> -u avnadmin -p --ssl-mode=REQUIRED defaultdb
```

```
CREATE TABLE stagiaires (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    ville VARCHAR(100) NOT NULL,
    photo VARCHAR(500) NOT NULL,
    date_inscription TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
DESCRIBE stagiaires;
```

Champ	Rôle
`id`	Identifiant généré automatiquement
`nom`	Nom du stagiaire

Champ	Rôle
`prenom`	Prénom du stagiaire
`ville`	Ville de résidence
`photo`	URL de l'image hébergée sur Cloudinary
`date_inscription`	Date d'inscription

2. Cloudinary

Créer un compte, ouvrir le tableau de bord et repérer le **Cloud name**, l'**API Key** et l'**API Secret**. La variable `CLOUDINARY_URL` a la forme :

```
cloudinary://<API_KEY>:<API_SECRET>@<CLOUD_NAME>
```

Les photos sont rangées dans le dossier `stagiaires` de la Media Library.

3. Dépendances (en local)

```
composer install
composer dump-autoload
```

- `composer install` installe les bibliothèques dans `vendor/`.
- `composer dump-autoload` actualise l'index des classes pour que PHP les trouve automatiquement.

`vendor/` n'est **pas** envoyé sur GitHub (`.gitignore`) : Vercel réinstalle les dépendances à partir de `composer.json` lors du déploiement. Si `vendor/` avait déjà été commité :

```
git rm -r --cached vendor
git add .gitignore composer.json composer.lock
git commit -m "Ignorer vendor et préparer le déploiement"
git push
```

4. Vercel : variables d'environnement

Settings → Environment Variables (environnement *Production*), puis redéployer :

Variable	Valeur
`DB_HOST`	hostname Aiven, ex. `mysql-xxxx-project-xxxx.c.aivencloud.com`
`DB_PORT`	port Aiven, ex. `15880`
`DB_NAME`	`defaultdb`

Variable	Valeur
`DB_USER`	`avnadmin`
`DB_PASSWORD`	mot de passe affiché dans Aiven
`CLOUDINARY_URL`	`cloudinary://API_KEY:API_SECRET@CLOUD_NAME`

Fonctionnement des fichiers PHP

`api/inscrire.php` (écriture)

1. Reçoit les données du formulaire (`$_POST` , `$_FILES`).
2. Vérifie les champs (obligatoires, 100 caractères max) et le fichier (type réel JPG/PNG/WEBP/GIF, 4 Mo max).
3. Envoie la photo à Cloudinary (`UploadApi::upload` , dossier `stagiaires`).
4. Récupère l'URL sécurisée `secure_url` (https).
5. Insère le stagiaire avec une **requête préparée** (protection contre l'injection SQL).
6. Renvoie une réponse JSON (`201` si succès, `422` si données invalides, `500` si erreur serveur).

`api/liste.php` (lecture)

Exécute `SELECT ... FROM stagiaires ORDER BY date_inscription DESC` et renvoie `{ succes, total, stagiaires: [...] }` en JSON. `liste.html` consomme cette API et charge chaque photo depuis son URL Cloudinary.

`config/db.php`

- `getenv()` récupère les paramètres configurés dans Vercel ;
- PDO établit la connexion à MySQL avec `utf8mb4` (caractères accentués) ;
- `ERRMODE_EXCEPTION` permet de détecter les erreurs SQL ;
- la connexion est chiffrée en TLS avec vérification du certificat `certs/ca.pem` (on ne désactive jamais cette vérification pour masquer une erreur).

Pourquoi ne pas enregistrer les photos dans le dossier `public` de Vercel ?

- Le système de fichiers d'une fonction Vercel est **en lecture seule et éphémère** : seul `/tmp` est accessible en écriture, et il est effacé entre deux exécutions. Une photo écrite sur le serveur disparaîtrait.
- Les fichiers du site sont figés **au moment du déploiement**. Un fichier ajouté ensuite n'est pas publié, et un nouveau déploiement repart du contenu de GitHub.
- Chaque requête peut être traitée par une instance différente : il n'existe pas de disque partagé entre elles.

On sépare donc les rôles : **Cloudinary** conserve durablement les images (et les sert via un CDN), **MySQL** ne stocke que leur URL.

Sécurité

- Les secrets (mot de passe MySQL, clé Cloudinary) restent dans les variables d’environnement de Vercel, jamais dans le code ni sur GitHub.
- `ca.pem` est un certificat public : il peut être versionné.
- Les données sont insérées avec des requêtes préparées et affichées avec `textContent` (pas d’injection HTML).